Kernels Pequenos em CNNs para Aceleração Winograd: Quanto Custam e Como Recuperar a Acurácia

Rafael Silva de Souza
Curso de Engenharia de Computação
Instituto de Informática
Universidade Federal do Rio Grande do Sul (UFRGS)
rafael.souza@inf.ufrgs.br

Mateus Grellert da Silva
Professor Adjunto, Orientador
Instituto de Informática
Universidade Federal do Rio Grande do Sul (UFRGS)
mateus.grellert@inf.ufrgs.br

***Resumo*—O algoritmo de Winograd faz convoluções com menos multiplicações, mas só compensa com kernels pequenos, como 2×2 e 3×3. Redes clássicas, como a AlexNet, usam kernels bem maiores. Este artigo mede quanto custa trocar esses kernels e como recuperar a acurácia perdida. Fizemos 209 treinos com uma única receita, no Tiny ImageNet e no CIFAR-100, e avaliamos cada rede em FP32 e em INT8. O kernel pesa menos do que se costuma supor. Passar dos kernels originais da AlexNet para 3×3 custa cerca de 1,5 ponto percentual (p.p.), um valor que ainda não se separa do acaso. Passar de 3×3 para 2×2 custa mais 2,5 p.p., mas essa perda desaparece quando as duas redes fazem o mesmo número de multiplicações. O stride da primeira camada pesa bem mais (mediana de +9,7 p.p.), e duas convoluções pequenas por estágio recuperam a perda. A quantização para INT8 quase não muda a acurácia. Uma AlexNet só com kernels 2×2 chega a cerca de 60,8%, contra 59,4% da VGG16, com um sexto dos parâmetros [uma seed; confirmar com as seeds 43 e 44].**

I. Introdução

O algoritmo de Winograd faz a mesma convolução com menos multiplicações [1]. Em duas dimensões, calcular um bloco de saída $m \times m$ com um kernel $r \times r$ pelo método direto usa m^2r^2 multiplicações. O método de Winograd usa $(m + r - 1)^2$. Com $m = 2$ e $r = 3$, são 16 multiplicações em vez de 36. O preço é a precisão numérica: o limite do erro cresce exponencialmente com o tamanho da convolução [2]. Por isso, esse método costuma ser usado com kernels pequenos.

Redes clássicas seguem outro caminho. A AlexNet [3] começa com kernels 11×11 e 5×5. A pergunta natural é quanto se perde ao trocar esses kernels por outros menores. A literatura não dá uma resposta única. A VGG mostrou que duas convoluções 3×3 seguidas enxergam a mesma região que uma 5×5 [4]. Trabalhos mais recentes, por outro lado, obtêm bons resultados com kernels grandes, usando convoluções *depthwise* [5], [6]. Esse tipo de convolução, porém, não recebe a aceleração de Winograd nas implementações que medimos [7].

Em um relatório anterior [7], comparamos variantes da AlexNet que diferiam em mais de um aspecto e treinaram por números diferentes de épocas. Isso atrapalha a leitura dos resultados, porque o treino pode pesar mais que a própria arquitetura [8], [9] e uma comparação justa precisa controlar o que muda [10]. Neste artigo, refazemos o estudo com uma única receita e mudamos uma coisa de cada vez. Nossas principais conclusões são:

- O kernel pesa menos que o stride da primeira camada, o BatchNorm, o Dropout e o pré-treino.
- O 2×2 perde para o 3×3 porque faz menos cálculo, não por causa do kernel: com o mesmo custo, os dois empatam.
- Duas convoluções pequenas por estágio recuperam a perda. Com esse corpo, o 2×2 passa a vencer o 3×3 e custa cerca de metade (por enquanto, uma seed).
- A quantização INT8 quase não muda a acurácia, seja qual for o kernel. Os grandes "ganhos" em INT8 vêm de treino extra.
- Boa parte dos ganhos atribuídos antes a blocos de compensação vinha das diferenças de treino.

II. Metodologia

A. Dados e receita de treino

Usamos o Tiny ImageNet [11]: imagens 64×64 de 200 classes. Separamos 90% do conjunto de treino para treinar e 10% para escolher a melhor época. Os resultados deste artigo são sempre do conjunto oficial de validação (10 mil imagens), usado como teste. Ele não participa de nenhuma escolha [12] e é o mesmo para todas as seeds. Os testes de robustez também usam o CIFAR-100 [13]. Reportamos a acurácia top-1.

Todas as redes seguem a mesma receita, inspirada na da AlexNet e da VGG [3], [4]: SGD com momentum 0,9, taxa de aprendizado 0,01, *weight decay* 5×10^{-4} e lotes de 128 imagens. A taxa de aprendizado cai em forma de cosseno até zero [14] ao longo de 500 épocas, sem parada antecipada. A perda usa *label smoothing* de 0,1 [15]. As imagens passam por deslocamento de até 4 pixels, espelhamento horizontal e AutoAugment [16], [17]. Os pesos das convoluções começam com a inicialização de He [18], que não favorece nenhum tamanho de kernel. Testes automáticos falham se alguma configuração sair dessa receita.

Tabela I
FATORES VARIADOS NOS EXPERIMENTOS

Fator	Opções
Kernels	11-5-3-3-3 (AlexNet original), 3×3, 2×2, alternado 3-2, 5×5; conv1 11×11 com corpo 3×3 ou 2×2
Layout	original (conv1 com stride 4, três *max-pools* 3×3, mapa final 1×1) ou 64 px (conv1 com stride 2, dois *max-pools* 2×2, mapa final 8×8)
Classificador	GAP ou FC + Dropout 0,5; FC sem Dropout só na comparação do Dropout
BatchNorm	com ou sem
Corpo	5 convoluções, 2 por estágio ou 2 por estágio + atalho residual
Stride da conv1	4, 2 ou 1
Pré-treino	do zero ou com pesos da ImageNet
Outras redes	VGG16, ResNet-18, MobileNetV2 e redes de compensação (Bottleneck, Fire e combinações)

B. Redes e fatores

Os experimentos formam três estudos (fases 11, 12 e 13 do projeto). O **estudo principal** tem 152 treinos e varia um fator de cada vez. Os **testes de robustez** têm 39 treinos e repetem as comparações com outra taxa de aprendizado, com ajuste fino de redes pré-treinadas e com outro conjunto de dados. O **estudo de combinações** tem 18 treinos novos e aproveita 6 do estudo principal, para testar se as técnicas que funcionaram se somam. A Tabela I resume os fatores.

Alguns termos aparecem nas figuras. A *conv1* é a primeira camada (algumas figuras dizem *stem*), e o *corpo* são as convoluções seguintes. A *cabeça* é o classificador final: GAP faz a média de cada canal e usa uma camada linear, e FC usa camadas totalmente conectadas. *Empilhado*, ou *2 por estágio*, quer dizer que cada convolução virou duas seguidas. O *skip residual* é o atalho de uma ResNet [19]. MACs são as multiplicações por imagem, nossa medida de custo. Toda convolução 2×2 de stride 1 usa preenchimento simétrico, para não deslocar a imagem a cada camada [20].

C. Quantização

Depois do treino em ponto flutuante (FP32), cada rede passa por 50 épocas de *Quantization-Aware Training* (QAT), um treino que simula a quantização [21], [22]. A taxa de aprendizado vai de 10^{-4} a 10^{-6} e o BatchNorm continua ativo [23]. Em seguida, a rede é convertida para inteiros de 8 bits (INT8) e avaliada em CPU. Ativações usam uma escala por tensor, e pesos, uma escala por canal.

D. Como comparamos

Uma comparação "A → B" usa pares de redes iguais em tudo menos em um fator. Reportamos a diferença B − A em pontos percentuais (p.p., ou "pp" nas figuras), com a mediana dos pares e um intervalo de confiança (IC) de 95% por *bootstrap*. Valores negativos indicam que B perdeu.

A maioria dos treinos usa a seed 42. Configurações importantes foram repetidas com as seeds 43 e 44, para medir a variação que vem só do acaso [24]. Duas redes iguais podem diferir, em FP32, em cerca de ±0,7 p.p. (AlexNet com BatchNorm), ±0,8 p.p. (AlexNet sem BatchNorm e VGG16) e ±1,0 p.p. (ResNet-18). Tratamos diferenças dentro dessa faixa como empate. Entre redes com uma só seed, diferenças abaixo de cerca de 1,5 p.p. são apenas uma indicação.

Marcamos os treinos cuja melhor época FP32 veio antes de 80% do total. Neles, o QAT parte de pesos obtidos com a taxa de aprendizado ainda alta, então a mudança para INT8 inclui treino extra. As figuras marcam esses casos com um anel, hachura ou asterisco ("INT8 inflado pelo QAT"). Eles nunca são removidos.

III. RESULTADOS

A. De onde vem a baixa acurácia inicial?

Vem mais da geometria da rede que do kernel. A AlexNet foi desenhada para imagens 224×224. Em 64×64, a conv1 com stride 4 e os três *max-pools* deixam um mapa final de 1×1, e a rede fica em 35,47%. Com os mesmos kernels e o layout de 64 px, ela chega a 44,00%. Com a imagem ampliada para 224 px, chega a 49,54%, mesmo sem nenhuma informação nova na imagem. São de 8,5 a 14 p.p. que não têm relação com o tamanho do kernel.

A Figura 1 mostra o efeito do stride: reduzir o da conv1 de 4 para 2 ganha de 6,4 a 13,0 p.p. nos oito pares, com mediana de +9,7 p.p., mas triplica o custo (+202% de MACs). Por isso, as comparações de kernel usam, a partir daqui, sempre o mesmo layout.

B. Quanto custa restringir o kernel?

Custa pouco. A Figura 2 e a Tabela II colocam o kernel ao lado dos outros fatores. Trocar 11-5-3-3-3 por 3×3 custa 1,5 p.p., com IC de −3,3 a +0,7, que inclui o zero, e reduz os MACs em 25%. Trocar 3×3 por 2×2 custa 2,5 p.p. (IC de −3,7 a −1,1) e reduz os MACs em 54%. O alternado 3-2 empata com o 3×3 e usa 24% menos MACs. Já BatchNorm, Dropout e pré-treino mudam a acurácia em 4 a 5 p.p.

O custo depende do classificador, como mostra a Figura 3. Com GAP e BatchNorm, o 3×3 nem chega a perder: fica em 53,81%, contra 53,07% do 11-5-3-3-3 (3 seeds cada), e o alternado 3-2 chega a 54,58%. Nessa configuração, o 2×2 custa 1,2 p.p. em relação ao 3×3 (52,58%). Com FC, Dropout e BatchNorm, a mesma troca custa 5,1 p.p. (seed 42).

A Figura 4 repete a comparação na VGG16. Com GAP e BatchNorm, o 2×2 custa 2,4 p.p. (56,84% contra 59,27%, 3 seeds cada). Com FC e Dropout, esse custo desaparece (59,17% contra 58,83%, 3 seeds). Com FC sem Dropout, o 2×2 fica 1,1 p.p. abaixo (uma seed). Na ResNet-18, o 2×2 custa 1,0 p.p. em relação ao 3×3 (3 seeds).

C. O 2×2 perde pelo kernel ou pelo custo?

Pelo custo. Para separar as duas coisas, alargamos a rede 2×2 até ela gastar o mesmo que a 3×3 e estreitamos a 3×3 até ela gastar o mesmo que a 2×2 (Figura 5a). Com o mesmo custo, as redes empatam: 53,83% contra 53,81% com 167M de MACs, e 52,58% contra 52,59% com 75M (3 seeds cada).

Kernels maiores, por outro lado, não compram acurácia. Com 5×5 em todas as convoluções, a rede custa 2,8 vezes

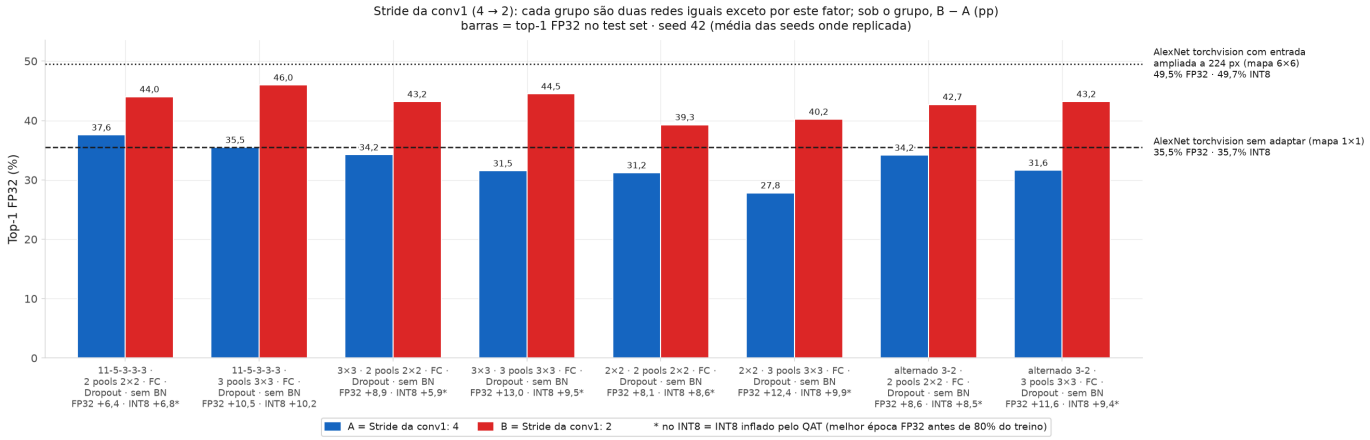


Figura 1. O stride da conv1 pesa mais que o kernel. Reduzir o stride de 4 para 2 (vermelho) ganha em todos os oito pares. As linhas horizontais são as referências da AlexNet sem adaptação e com a imagem ampliada para 224 px.

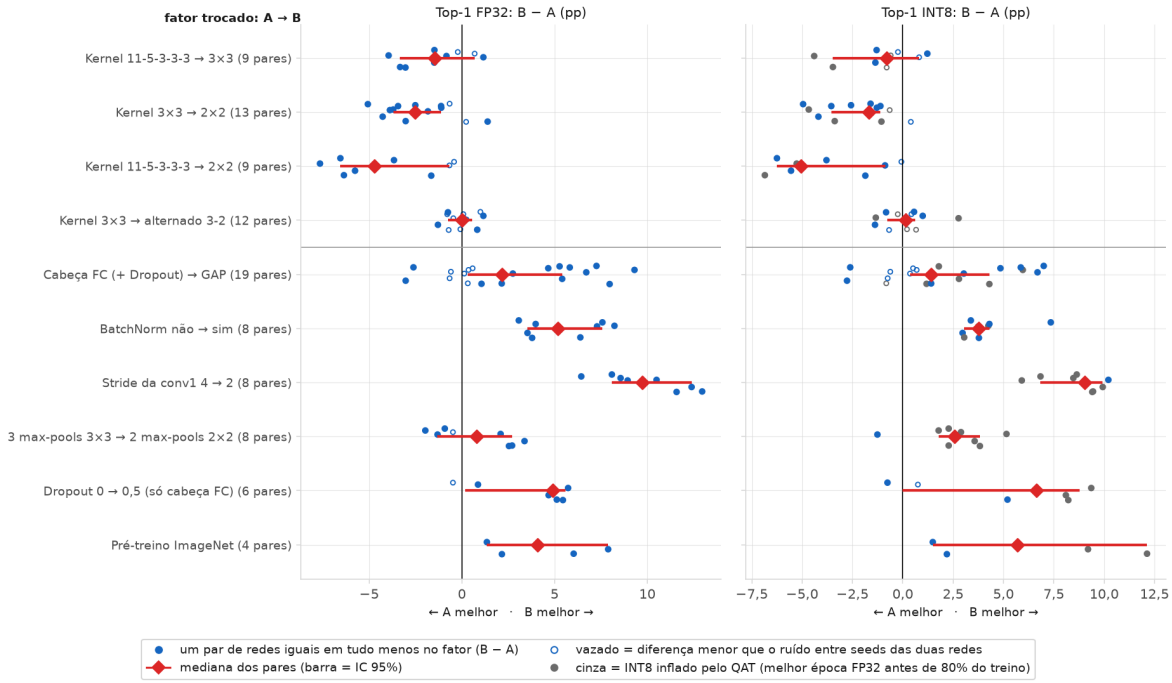


Figura 2. Efeito de trocar um fator de cada vez, em FP32 e em INT8 (seed 42, comparações com 3 ou mais pares). Cada ponto é um par de redes, e o losango é a mediana. As quatro primeiras linhas são de kernel: note como ficam perto do zero em relação às outras. Pontos vazados estão dentro da variação entre seeds, e os cinzas têm INT8 inflado pelo QAT.

o 3×3 e chega a 49,25%, ou 4,6 p.p. abaixo dele (3 seeds, Figura 5b). Com GAP e BatchNorm, gastar mais só compensou como largura, não como kernel maior. A exceção é o classificador FC + Dropout, em que o 5×5 chega a 52,14% (uma seed), perto dos 52,23% do 11-5-3-3-3.

A Figura 6 mostra onde o 2×2 custa na VGG16. Colocá-lo em um único bloco quase não muda nada: 59,54%, 59,34%, 59,05% e 59,18% nos blocos 1, 2, 3 e 5 (3 seeds), contra 59,27% com 3×3 em todos. O bloco 4 é o único que parece custar algo (58,56%). Essa diferença de 0,71 p.p. ainda está dentro da variação, mas se repete nas três seeds. A perda maior só aparece quando todos os blocos usam 2×2.

D. Dá para recuperar a perda sem um corpo de kernel grande?

Sim, de duas formas. A primeira é manter só a conv1 11×11 e usar 2×2 no resto (Figura 7). Essa rede chega a 55,93% em FP32 e 56,11% em INT8 (3 seeds) com 98M de MACs. É mais que o 3×3 (53,81%, 167M) e que os kernels originais (53,07%, 240M). Como a conv1 tem stride 2, ela já ficaria fora de um acelerador de Winograd de qualquer forma, então o kernel grande não tira nada dele.

Tabela II
EFEITO DE CADA FATOR: MEDIANA DE B − A ENTRE PARES DE REDES IGUAIS NO RESTO (SEED 42)

A → B	Pares	FP32 (p.p.)	IC 95%	B acima de A	INT8 (p.p.)	MACs	Parâmetros
Kernel 11-5-3-3-3 → 3×3	9	−1,5	[−3,3; +0,7]	2 de 9	−0,8	−25%	−1%
Kernel 3×3 → 2×2	13	−2,5	[−3,7; −1,1]	2 de 13	−1,7	−54%	−20%
Kernel 11-5-3-3-3 → 2×2	9	−4,7	[−6,5; −0,7]	0 de 9	−5,1	−56%	−5%
Kernel 3×3 → alternado 3-2	12	0,0	[−0,7; +0,5]	6 de 12	+0,1	−24%	−6%
3×3 → dois 3×3 por estágio	2	+5,3	[+2,4; +8,1]	2 de 2	+5,4	+148%	+65%
2×2 → dois 2×2 por estágio	2	+8,0	[+4,9; +11,2]	2 de 2	+8,2	+133%	+62%
Conv1 3×3 → 11×11, corpo 3×3	2	+1,3	[+1,0; +1,6]	2 de 2	+1,4	+12%	0%
Conv1 2×2 → 11×11, corpo 2×2	2	+3,7	[+3,3; +4,1]	2 de 2	+3,9	+24%	+1%
FC + Dropout → GAP	19	+2,1	[+0,3; +5,4]	15 de 19	+1,4	−25%	−96%
BatchNorm: não → sim	8	+5,2	[+3,5; +7,6]	8 de 8	+3,8	0%	0%
Stride da conv1: 4 → 2	8	+9,7	[+8,1; +12,4]	8 de 8	+9,0	+202%	+50%
3 *max-pools* 3×3 → 2 *max-pools* 2×2	8	+0,8	[−1,3; +2,7]	4 de 8	+2,6	+57%	+88%
Dropout: 0 → 0,5 (só FC)	6	+4,9	[+0,2; +5,6]	5 de 6	+6,7	0%	0%
Pré-treino ImageNet	4	+4,1	[+1,4; +7,9]	4 de 4	+5,7	0%	0%

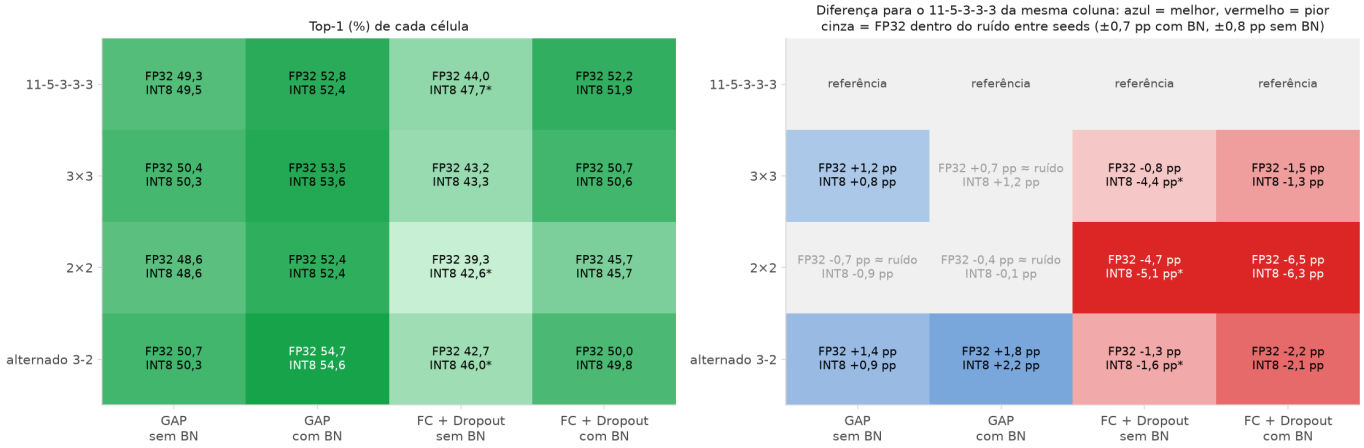


Figura 3. Kernel, classificador e BatchNorm na AlexNet de layout 64 px. À esquerda, a acurácia de cada combinação; à direita, a diferença para o 11-5-3-3-3 da mesma coluna. Com GAP (duas primeiras colunas), os kernels pequenos ficam perto do original ou acima dele. Cinza: dentro da variação entre seeds; asterisco: INT8 inflado pelo QAT. Para comparar: a AlexNet sem adaptação fica em 35,5% em FP32, e com imagem ampliada para 224 px, em 49,5%. Seed 42.

A segunda forma é usar duas convoluções pequenas por estágio. Isso ganha 5,3 p.p. com 3×3 e 8,0 p.p. com 2×2, com 130 a 150% mais MACs (Tabela II, dois pares cada). Com GAP e BatchNorm, a rede 2×2 empilhada (57,25%, 201M) fica até acima da 3×3 empilhada (55,87%, 451M), com uma seed cada.

O campo receptivo efetivo ajuda a entender a diferença [25]. A Figura 8 mostra quanto cada pixel influencia o centro da última camada. Empilhar amplia a região usada: a área com metade da influência vai de 17,8% para 29,8% da imagem no 2×2, e de 23,5% para 33,1% no 3×3. Já a conv1 11×11 quase não amplia essa área (18,5% contra 17,8%), embora melhore a acurácia. Seu ganho, portanto, não parece vir de enxergar mais da imagem. Não testamos a causa.

E. A quantização INT8 atrapalha?

Quase nada, e o kernel não muda isso (Figura 9). Nos 126 treinos do estudo principal sem a marca de melhor época cedo, a diferença entre INT8 e FP32 vai de −0,38 a +0,58 p.p., com mediana de +0,04 p.p. Nos 26 treinos marcados, ela vai de +0,56 a +6,69 p.p. Esse aumento não é efeito da quantização: o QAT continua treinando uma rede que ainda não tinha terminado de convergir, agora com uma taxa de aprendizado baixa. No estudo de combinações, nenhum treino recebeu a marca, e a diferença vai de −0,44 a +0,77 p.p.

Acurácia igual, porém, não quer dizer respostas iguais [26]. A Figura 10 conta as imagens que mudam entre certo e errado. A maioria das redes troca de 2% a 7% das imagens de teste, em quantidades parecidas nas duas direções. As redes marcadas chegam, em geral, a 9–14%, e quase sempre ganham no saldo.

Há uma exceção a acompanhar: a MobileNetV2 pré-treinada. Com ajuste fino, ela perde 1,49 p.p. (taxa de aprendizado 10^{-4}) e 1,16 p.p. (10^{-3}) do FP32 para o INT8, quase tudo já no QAT. Fora ela, nenhum treino sem marca dos dois primeiros estudos perde mais de 0,38 p.p. Ela é a única rede pré-treinada com convoluções *depthwise*, mas não investigamos a causa.

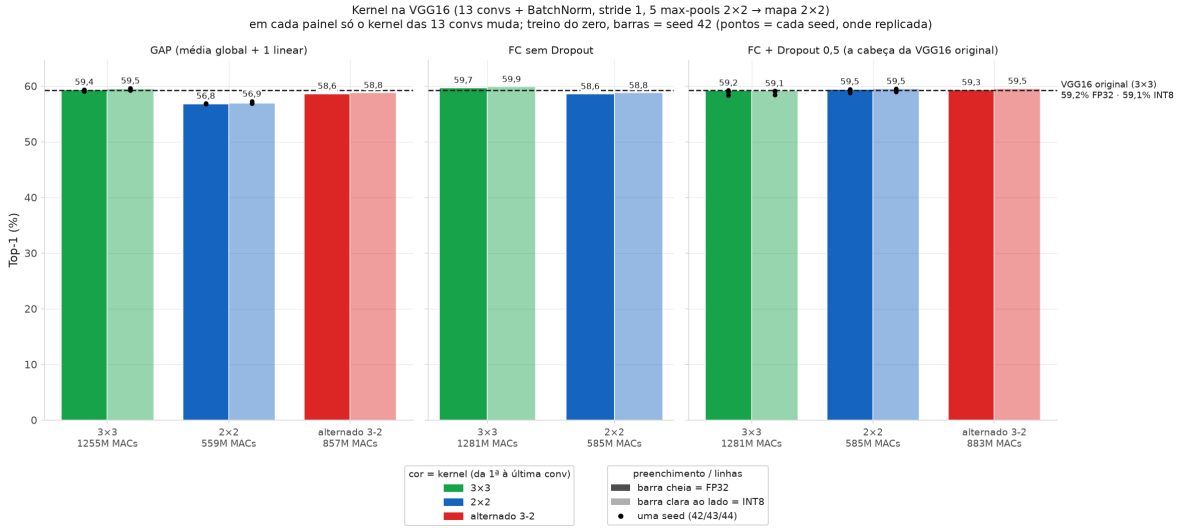


Figura 4. Kernel na VGG16. Em cada painel, só o kernel das 13 convoluções muda. O 2×2 perde com GAP e, por menos, com FC sem Dropout (uma seed). Com FC + Dropout, empata.

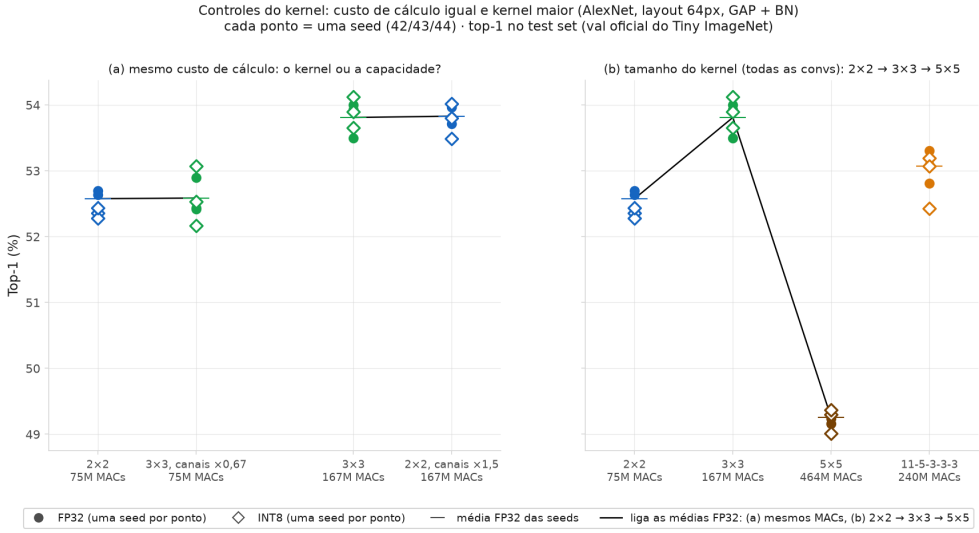


Figura 5. (a) Com o mesmo custo em MACs, 2×2 e 3×3 empatam. (b) Um kernel maior não ajuda: o 5×5 custa mais e fica bem abaixo do 3×3. Cada ponto é uma seed. AlexNet de layout 64 px, GAP e BatchNorm.

F. Quanto da rede cabe em um acelerador Winograd?

As Figuras 11 e 12 comparam acurácia e custo. A linha tracejada liga as redes que nenhuma outra supera em acurácia e custo ao mesmo tempo. Na parte barata, ela passa por redes de compensação, como a Bottleneck. Na parte intermediária, passa pela conv1 11×11 com corpo 2×2 e pela 2×2 empilhada. Na parte cara, passa pela VGG16.

Barato não significa adequado para Winograd. A Tabela III mostra a parcela dos MACs em convoluções densas de stride 1, com kernel 2×2 ou 3×3. As AlexNets de kernel pequeno e a VGG16 ficam perto de 100%. A TinyMobileNetV2 e a MobileNetV2 ficam em torno de zero, e a FinalBottleneck-Residual, em 28%. Essas redes são eficientes, mas gastam a maior parte do cálculo em convoluções 1×1, *depthwise* ou de stride maior que 1, que o acelerador não aproveita [7]. A busca por uma rede eficiente e a busca por uma rede para Winograd levam, portanto, a escolhas diferentes.

G. Os resultados valem em outras condições?

Sim. A Figura 13 resume os cenários. Trocar 3×3 por 2×2 custa de 0,7 a 2,4 p.p. em FP32 em todos eles e quase nada quando os MACs são iguais. Com metade e com o dobro da taxa de aprendizado, a ordem das três redes testadas se mantém: 3×3, 11-5-3-3-3 e 2×2 (52,27%, 52,15% e 51,57%; 54,22%, 53,89% e 53,22%; uma seed cada). Trocar os kernels originais por 3×3 empata ou melhora nos cenários com AlexNet. A exceção é a ResNet-18 (−0,95 p.p., uma seed).

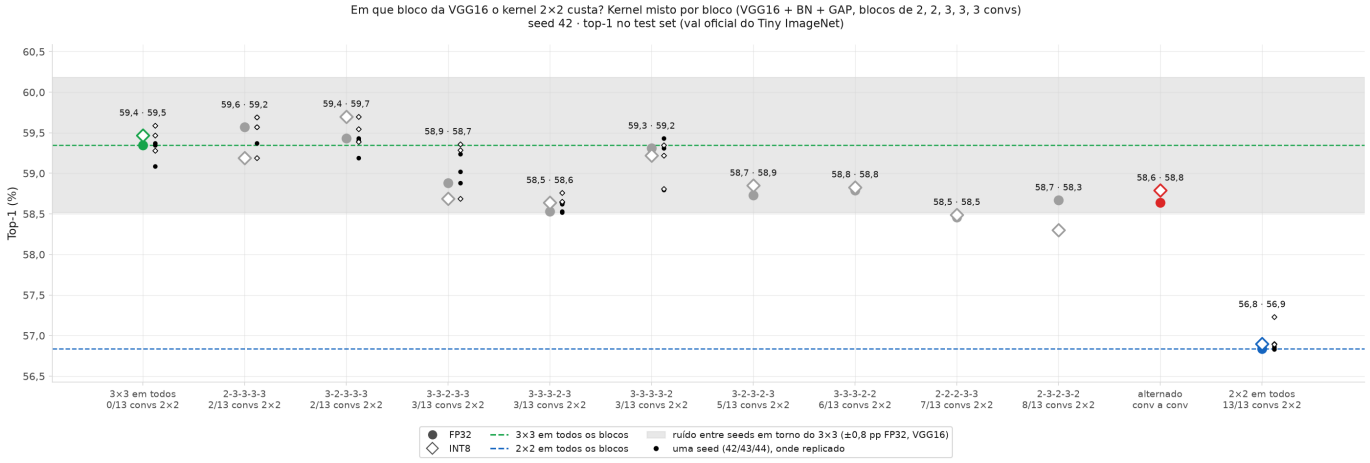


Figura 6. VGG16 com um kernel por bloco. Os padrões leem-se bloco a bloco: 3-3-3-2-3 tem 2×2 só no bloco 4. A faixa cinza é a variação entre seeds em torno do 3×3. Só a rede com 2×2 em todos os blocos sai claramente dela.

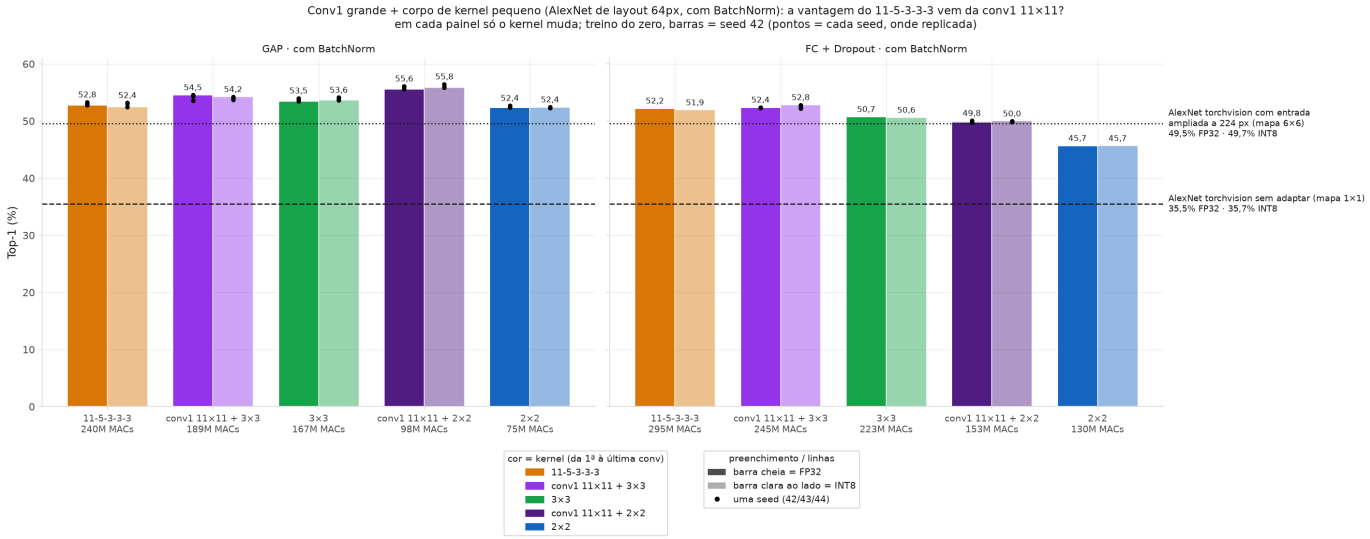


Figura 7. Conv1 grande com corpo de kernel pequeno. Com GAP (à esquerda), a conv1 11×11 com corpo 2×2 é a melhor das cinco e a segunda mais barata. Barra cheia: FP32; barra clara: INT8; pontos: cada seed.

No CIFAR-100 (3 seeds), a AlexNet com alternado 3-2 chega a 63,42%, com 3×3 a 63,34%, com 11-5-3-3-3 a 63,00% e com 2×2 a 62,09%. Na VGG16, o 3×3 chega a 71,08%, o alternado a 70,99% e o 2×2 a 70,31%. A ordem dos kernels é a mesma do Tiny ImageNet nas duas famílias (Figura 14), com τ de Kendall igual a $+1{,}00$ [27]. Como são só quatro e três redes, e algumas ficam muito próximas, isso é um sinal de consistência, não uma prova.

O pré-treino merece uma observação. Com a receita única, a taxa de 0,01 apaga boa parte dele. Com ajuste fino de verdade [28], a taxa de 10^{-3} supera a de 10^{-4} em todas as redes testadas em 64 px, e a VGG16 chega a 67,27%. O ganho sobre a mesma rede treinada do zero vai de $+5{,}5$ a $+14{,}9$ p.p. Esse resultado não responde à pergunta sobre kernels, porque os pesos pré-treinados só existem para os kernels originais de cada rede.

H. Os ganhos se somam?

Em parte. Testamos todas as combinações de kernel do corpo (3×3, 2×2), conv1 (o mesmo kernel ou 11×11), corpo (5 convoluções, 2 por estágio, 2 por estágio com atalho) e stride da conv1 (2 ou 1), na AlexNet com GAP e BatchNorm. São 24 redes, com a seed 42. A Figura 15 e a Tabela IV mostram o resultado.

O stride 1 é o maior fator e nunca falha, mas custa cerca de quatro vezes mais MACs. Dois kernels por estágio vêm em seguida e também nunca falham. O atalho residual fica dentro da variação entre seeds, e seu sinal muda entre os pares. A conv1 11×11 só ajuda no corpo de 5 convoluções (3 de 4 pares). Com dois kernels por estágio, sua mediana é de $-0{,}20$ p.p., ou seja, a segunda convolução por estágio já entrega o que ela entregava.

O resultado mais curioso é o do kernel do corpo. Com 5

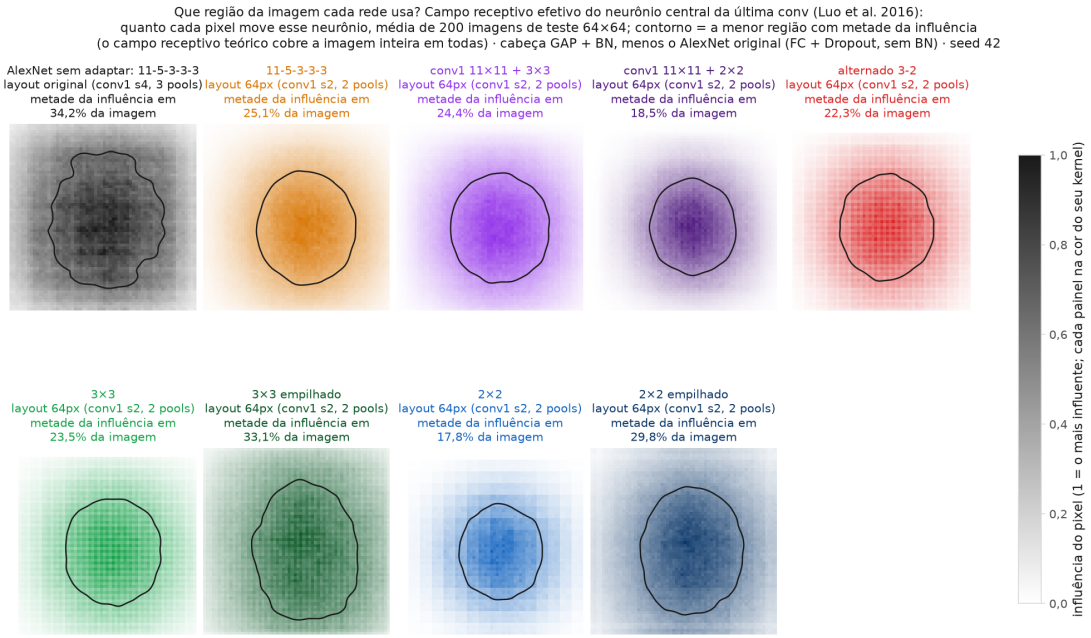


Figura 8. Que região da imagem cada rede usa (média de 200 imagens de teste, seed 42). O contorno marca a menor região com metade da influência. Compare, na segunda linha, 2×2 com 2×2 empilhado e 3×3 com 3×3 empilhado.

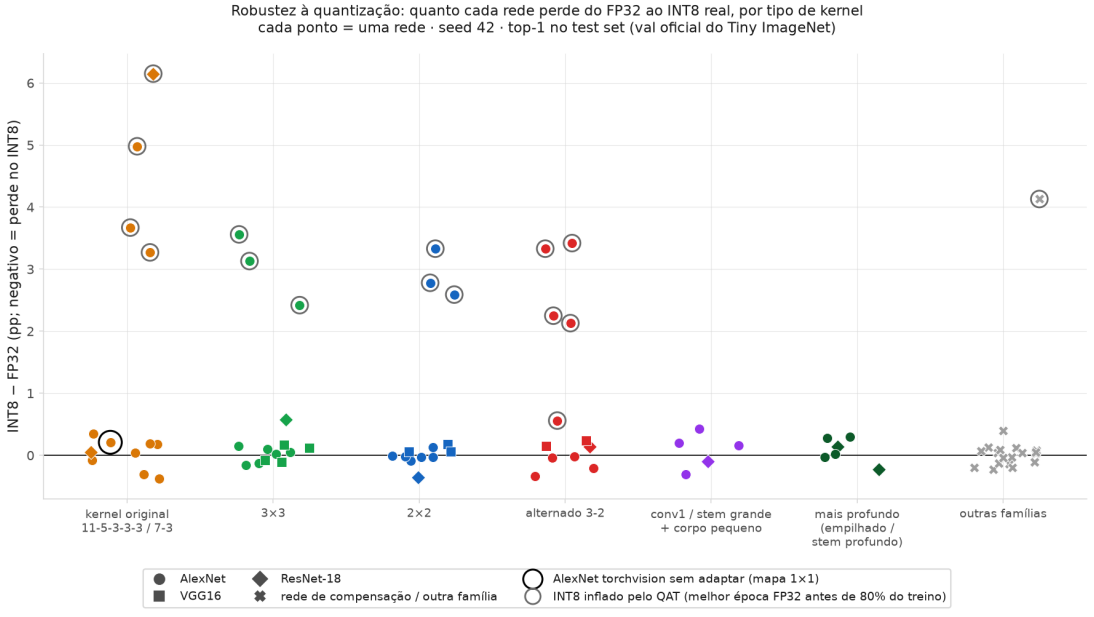


Figura 9. Quanto cada rede muda do FP32 para o INT8, por tipo de kernel (seed 42). Sem a marca do anel, todas as redes ficam perto de zero, seja qual for o kernel. Os pontos altos, com anel, são treino extra do QAT.

convoluções, a troca de 3×3 por 2×2 varia de −1,28 a +1,09 p.p. Com dois kernels por estágio, o 2×2 vence nos 8 pares (de +0,47 a +1,82 p.p., mediana de +1,4), com cerca de metade dos MACs.

Os dois maiores ganhos não se somam por inteiro. Partindo do 3×3 com stride 2 (53,50%), os ganhos separados de empilhar (+2,37) e de usar stride 1 (+4,16) levariam a 60,03%. O valor medido é 58,53%. No 2×2, a soma levaria a 61,24% e o valor medido é 60,35%. O corpo 2×2 perde menos nessa combinação.

As melhores redes usam kernels 2×2, dois por estágio, e stride 1. Com atalho, a rede chega a 60,75% em FP32 e

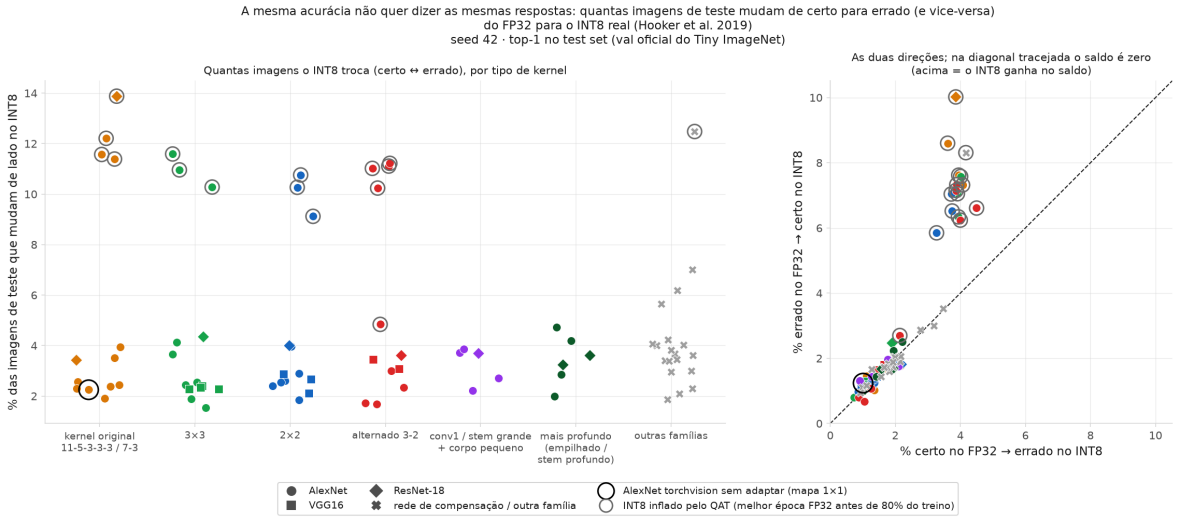


Figura 10. Imagens que mudam entre certo e errado do FP32 para o INT8 (seed 42). À direita, sobre a diagonal, o saldo é zero. Só as redes com anel ficam claramente acima dela.

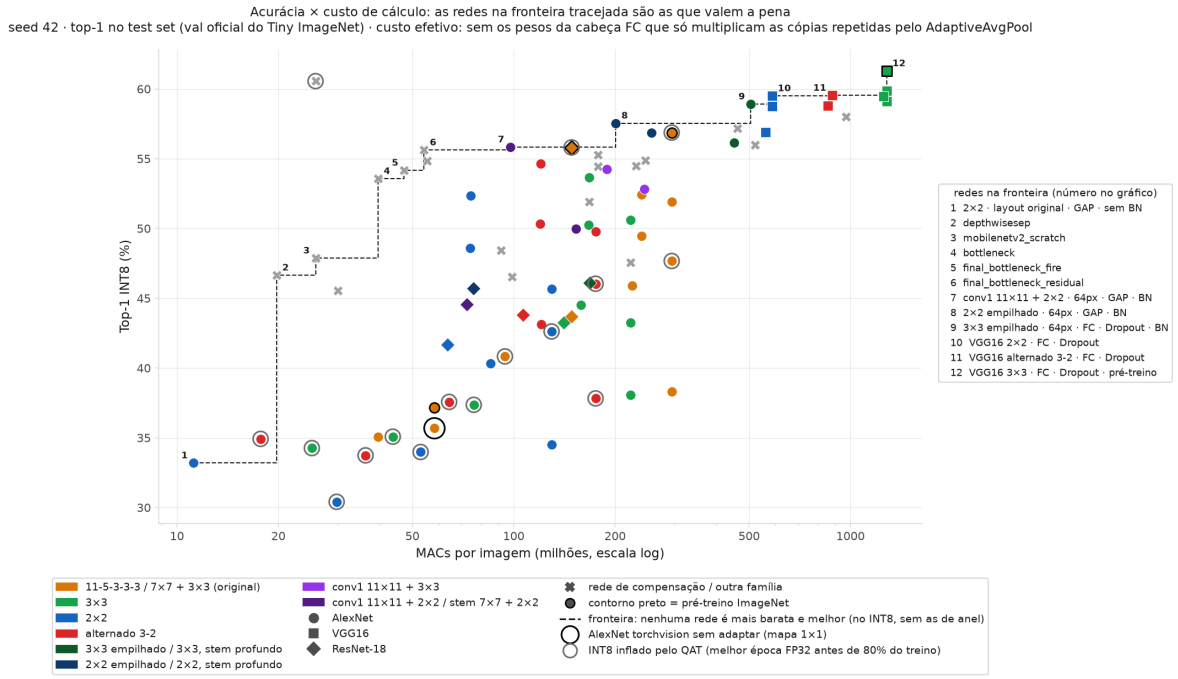


Figura 11. Acurácia INT8 e custo em MACs (estudo principal, seed 42). A linha tracejada liga as redes que nenhuma outra supera nos dois critérios ao mesmo tempo; as redes com anel ficam fora dela, porque seu INT8 está inflado pelo QAT.

60,46% em INT8 (2,52M de parâmetros, 862M de MACs). Sem atalho, chega a 60,35% e 60,79% (2,33M, 803M). A VGG16 3×3 com GAP e BatchNorm fica em 59,35% e 59,47%, com 14,82M de parâmetros e 1255M de MACs (de 59,09% a 59,37% em 3 seeds). As duas AlexNets ficam de 1,0 a 1,4 p.p. acima dela, com cerca de um sexto dos parâmetros e dois terços dos MACs. Como essas redes foram as melhores entre 24, a seed 42 pode estar favorecendo as duas. Por isso, repetimos as duas com as seeds 43 e 44. [Seeds 43 e 44 em andamento. Atualizar esta seção com a média das 3 seeds, a comparação do atalho por seed e a comparação com a VGG16 antes de tirar conclusões.]

IV. Discussão

A. *Como desenhar uma rede para Winograd*

Os resultados apontam para algumas escolhas. Primeiro, acerte a geometria e o treino: stride baixo na conv1 para imagens pequenas, BatchNorm e GAP. Esses fatores pesam mais que o kernel. Segundo, não aumente o kernel para ganhar acurácia: gastar a mais em largura ou em uma segunda

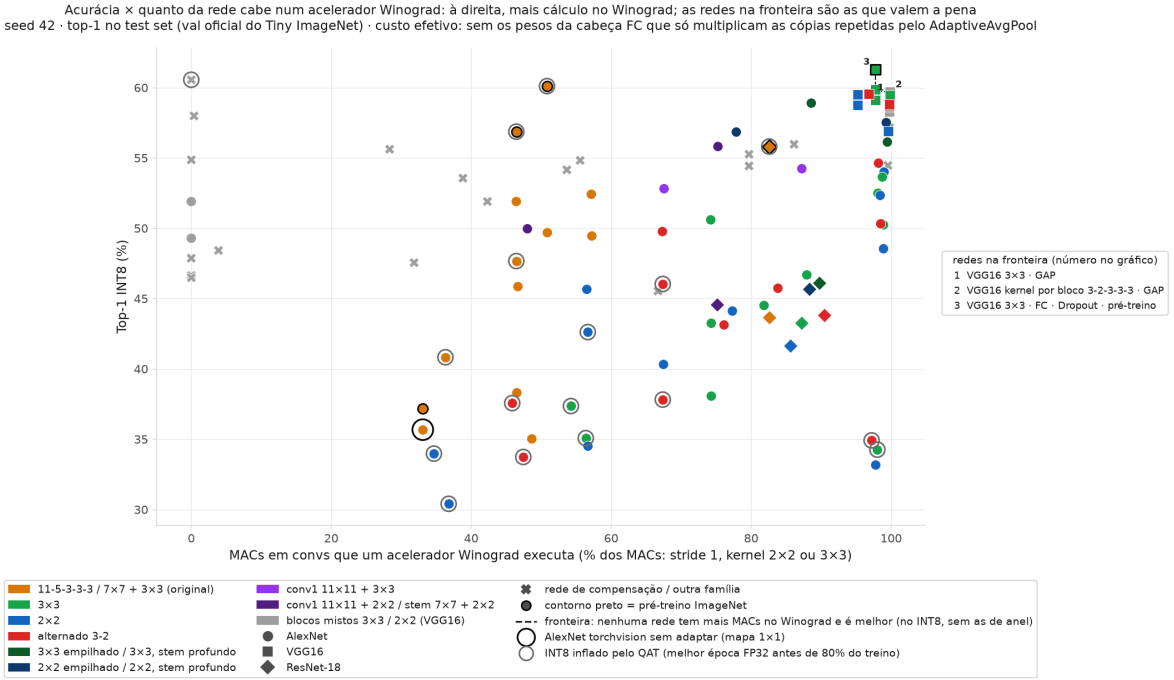


Figura 12. Acurácia INT8 e parcela dos MACs em convoluções que um acelerador Winograd pode executar (stride 1, kernel 2×2 ou 3×3, sem grupos). Sem contar as redes com anel, as mais precisas, no canto superior direito, cabem quase inteiras. A conta usa só a geometria das camadas.

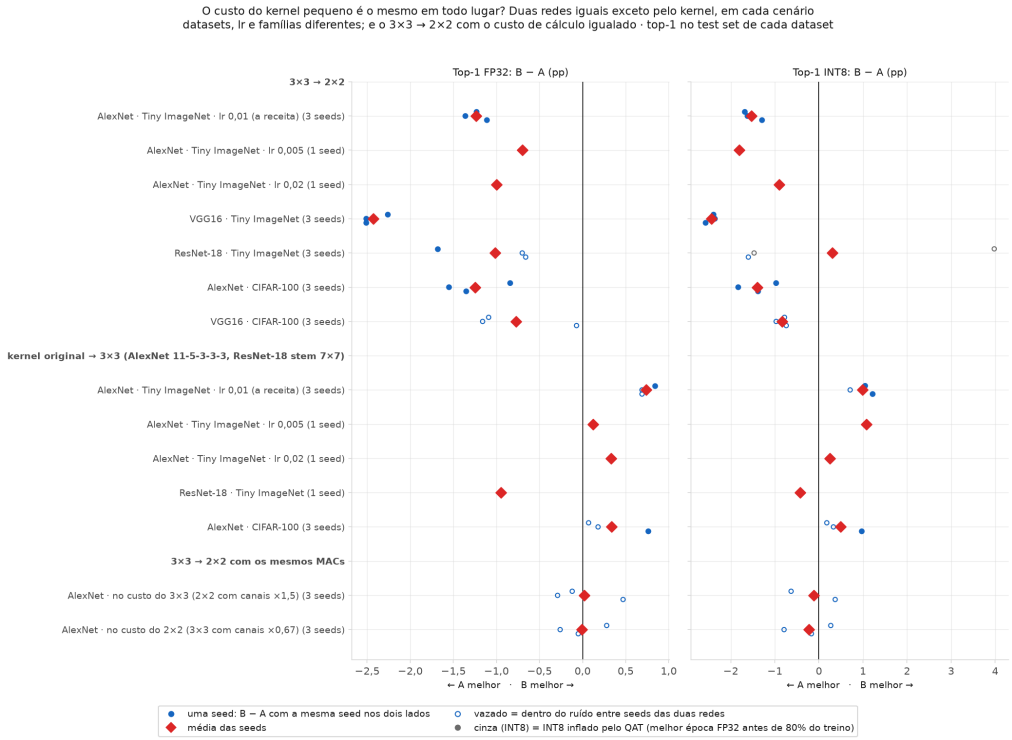


Figura 13. O efeito do kernel em cada cenário: duas redes iguais exceto pelo kernel. O custo do 2×2 aparece em todas as linhas do primeiro grupo e some no último, em que os MACs são iguais. O grupo do meio compara os kernels originais com 3×3. Um ponto muito à direita no INT8 da ResNet-18 é um caso de INT8 inflado pelo QAT.

convolução por estágio rende mais. Terceiro, evite redes em que a maior parte do cálculo fica em convoluções *depthwise* ou 1×1 se o alvo é Winograd. Quarto, não trate a quantização como obstáculo: o INT8 quase não muda a acurácia. O stride

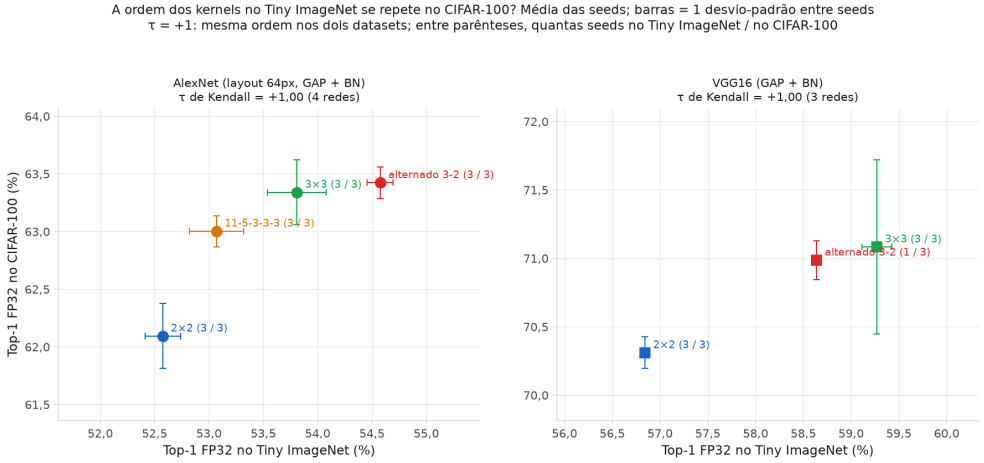


Figura 14. A ordem dos kernels no Tiny ImageNet se repete no CIFAR-100. Cada ponto é a média das seeds, e as barras mostram um desvio-padrão entre seeds.

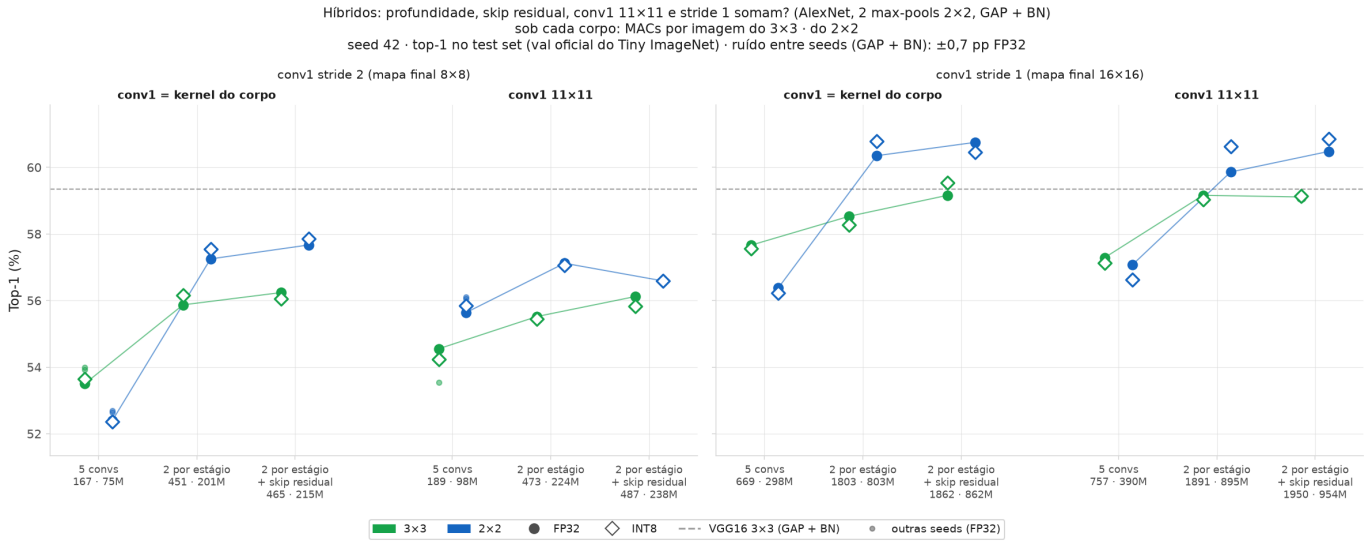


Figura 15. Todas as combinações (seed 42). Da esquerda para a direita, em cada grupo, o corpo fica mais profundo. O painel da direita (stride 1) fica acima do da esquerda (stride 2), e o azul (2×2) passa o verde (3×3) quando há dois kernels por estágio. A linha tracejada é a VGG16 3×3. Sob cada corpo estão os MACs do 3×3 e do 2×2.

1 é o fator mais caro: ele quadruplica os MACs, e cabe a cada projeto decidir se vale esse custo.

A escolha entre 2×2 e 3×3 depende do hardware. Em acurácia, com dois kernels por estágio, o 2×2 vence e custa cerca de metade. Em um estudo paralelo do grupo com 14 redes em FPGA, porém, as transformações testadas são do tipo F(m,3), feitas para kernels 3×3. O kernel 2×2 fica fora delas. Nesse estudo, o tamanho da transformação é o que define o custo em acurácia: F(2,3) fica praticamente igual ao FP32 (+0,05 p.p., mediana), F(4,3) perde 5,1 p.p. e F(6,3) perde 18,9 p.p. Assim, um acelerador com F(2,3) e redes 3×3 é hoje a opção sem custo medido. Usar 2×2 exige um acelerador preparado para esse kernel.

B. O que muda em relação ao trabalho anterior

Sob a mesma receita, boa parte das diferenças entre redes de compensação desaparece. No relatório anterior [7], um atalho residual em um bloco Fire [29] parecia ganhar mais de 6 p.p., mas os treinos seguiam protocolos diferentes, inclusive no número de épocas. Agora, Fire fica em 54,57% e FireBypass em 55,19%, uma diferença de 0,62 p.p., dentro da variação. As redes Fire, FireBypass e as combinações com Fire e Bottleneck ficam a menos de 1,5 p.p. umas das outras (de 54,40% a 55,83%, seed 42). O estudo de combinações aponta na mesma direção: o atalho não mostra ganho mensurável. Isso não invalida esses blocos, que continuam baratos, mas muda o motivo para usá-los.

As demais fases do projeto entram só como contexto, porque usaram protocolos diferentes. Medições anteriores em GPU

Tabela III
ACURÁCIA, CUSTO E PARCELA DOS MACS EM CONVOLUÇÕES DENSAS DE STRIDE 1 COM KERNEL 2×2 OU 3×3 (SEED 42)

Rede	FP32 (%)	INT8 (%)	Parâm. (M)	MACs (M)	Winograd (%)
2×2 + atalho, stride 1	60,75	60,46	2,52	862	92,9
2×2 empilhada, stride 1	60,35	60,79	2,33	803	99,6
VGG16 3×3	59,35	59,47	14,82	1255	99,8
TinyMobileNetV2	57,96	58,00	1,51	969	0,4
2×2 empilhada	57,25	57,55	2,33	201	99,2
SmallKernel + BN	57,20	57,17	1,60	461	99,7
FinalBottleneckResidual	55,83	55,63	0,57	54	28,3
Conv1 11×11 + corpo 2×2	55,64	55,84	1,08	98	75,2
FireBypass	55,19	55,27	0,52	178	79,7
Fire	54,57	54,44	0,52	178	79,7
Bottleneck	53,71	53,57	0,39	40	38,8
3×3	53,50	53,65	2,30	167	98,7
11-5-3-3-3	52,81	52,43	2,52	240	57,1
2×2	52,39	52,36	1,05	75	98,4
MobileNetV2 (do zero)	47,83	47,88	2,48	26	0,0
DepthwiseSep	46,53	46,66	0,31	20	0,0

Tabela IV
EFEITO DE CADA FATOR NAS 24 COMBINAÇÕES: MEDIANA DE B − A (SEED 42, P.P.)

A → B	FP32	INT8	B acima	Faixa FP32
Stride da conv1: 2 → 1	+3,03	+3,40	12 de 12	+1,43 a +4,16
5 convs → 2 por estágio	+2,12	+2,21	8 de 8	+0,87 a +4,86
Corpo 3×3 → 2×2	+1,23	+1,50	9 de 12	−1,28 a +1,82
2 por estágio → + atalho	+0,41	+0,16	6 de 8	−0,53 a +0,63
Conv1 → 11×11	−0,12	−0,19	4 de 12	−1,08 a +3,25

indicaram que a aceleração Winograd depende de convoluções densas de stride 1. Em detecção, a compensação estrutural se transferiu só em parte. Em atenção local, o tamanho da janela se comportou como o tamanho do kernel [7].

C. Limitações

A maioria das comparações usa uma só seed. A receita é a mesma para todas as redes, mas não é a melhor para cada uma, e uma receita pode mudar a ordem entre arquiteturas [8], [9]. Os dados são de baixa resolução: 64 px e 32 px, e as imagens de 224 px são apenas ampliações. A parcela de MACs mede só a geometria das camadas, e não medimos a vazão em um acelerador real. Não comparamos latência entre redes, porque os treinos rodaram em máquinas diferentes (RTX 4090, RTX 4060 Ti e GTX 1080 Ti). Essa diferença de máquina faz parte da variação entre execuções e também afeta a seed 42 do estudo de combinações: as duas melhores redes foram treinadas em GPUs diferentes.

V. CONCLUSÃO

Restringir o kernel custa pouco. Trocar os kernels originais da AlexNet por 3×3 custa cerca de 1,5 p.p., e com GAP e BatchNorm nem isso. O 2×2 custa mais, porém esse custo vem de fazer menos cálculo: com os mesmos MACs, ele empata com o 3×3. A geometria da rede, o BatchNorm, o Dropout e o pré-treino pesam mais, e a quantização INT8 quase não pesa. Duas convoluções pequenas por estágio recuperam a perda, e com elas o 2×2 passa a vencer o 3×3 pela metade do custo. Essas conclusões se repetem com outras taxas de aprendizado e no CIFAR-100. Para quem projeta redes para aceleradores Winograd, a mensagem é simples: não é preciso manter kernels grandes no corpo da rede para ter boa acurácia.

VI. TRABALHOS FUTUROS

O passo seguinte é concluir as seeds 43 e 44 das melhores redes. Depois, queremos reavaliar todas as redes no mesmo tipo de máquina, para comparar latência; medir a vazão em um acelerador real; testar blocos Bottleneck e Fire sobre o melhor corpo, para ver se reduzem o custo sem perder acurácia; investigar a perda da MobileNetV2 pré-treinada em INT8; e repetir as comparações principais em imagens de resolução maior.

AGRADECIMENTOS

Experimentos utilizaram recursos da infraestrutura PCAD (http://pcad.inf.ufrgs.br).

REFERÊNCIAS

[1] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.

[2] B. Barabasz, A. Anderson, K. M. Soodhalter, and D. Gregg, "Error analysis and improving the accuracy of Winograd convolution for deep neural networks," *ACM Transactions on Mathematical Software*, vol. 46, no. 4, Art. 37, 2020.

[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[4] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations (ICLR)*, 2015.

[5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.

[6] X. Ding, X. Zhang, Y. Zhou, J. Han, G. Ding, and J. Sun, "Scaling up your kernels to 31x31: Revisiting large kernel design in CNNs," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.

[7] R. S. de Souza and M. G. da Silva, "Restrição do tamanho de kernel em CNNs para aceleração Winograd," relatório de pesquisa, Instituto de Informática, UFRGS, 2026.

[8] I. Bello *et al.*, "Revisiting ResNets: Improved training and scaling strategies," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[9] R. Wightman, H. Touvron, and H. Jégou, "ResNet strikes back: An improved training procedure in timm," *arXiv preprint arXiv:2110.00476*, 2021.

[10] I. Radosavovic, J. Johnson, S. Xie, W.-Y. Lo, and P. Dollár, "On network design spaces for visual recognition," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[11] Y. Le and X. Yang, "Tiny ImageNet visual recognition challenge," Stanford CS231N Report, 2015.

[12] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, 2010.

[13] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.

[14] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, "Bag of tricks for image classification with convolutional neural networks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019.

[15] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
[16] P. Chrabaszcz, I. Loshchilov, and F. Hutter, "A downsampled variant of ImageNet as an alternative to the CIFAR datasets," *arXiv preprint arXiv:1707.08819*, 2017.
[17] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning augmentation strategies from data," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 113–123.
[18] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2015.
[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
[20] S. Wu, G. Wang, P. Tang, F. Chen, and L. Shi, "Convolution with even-sized kernels and symmetric padding," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
[21] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
[22] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," *arXiv preprint arXiv:2004.09602*, 2020.
[23] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.
[24] X. Bouthillier *et al.*, "Accounting for variance in machine learning benchmarks," in *Proc. Machine Learning and Systems (MLSys)*, 2021.
[25] W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
[26] S. Hooker, A. Courville, G. Clark, Y. Dauphin, and A. Frome, "What do compressed deep neural networks forget?" *arXiv preprint arXiv:1911.05248*, 2019.
[27] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, no. 1–2, pp. 81–93, 1938.
[28] H. Li *et al.*, "Rethinking the hyperparameters for fine-tuning," in *International Conference on Learning Representations (ICLR)*, 2020.
[29] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," *arXiv preprint arXiv:1602.07360*, 2016.

APÊNDICE

A. Figuras de apoio

As figuras a seguir trazem o detalhe por trás do texto: as curvas de treino, que mostram que as redes param de melhorar antes do fim das épocas, e as barras de cada fator da Tabela II.

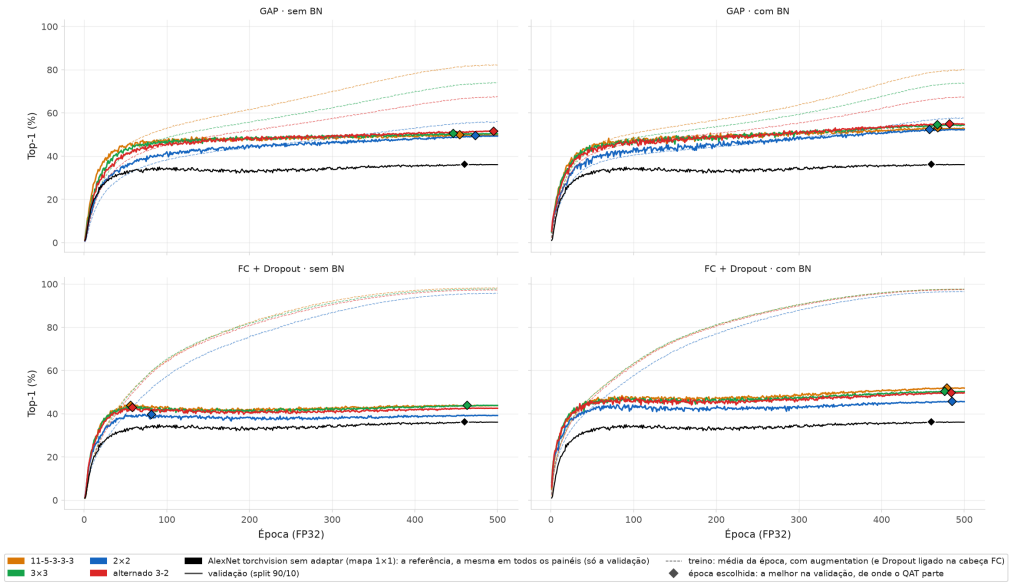


Figura 16. Curvas de treino em FP32 (AlexNet de layout 64 px, seed 42), medidas nos 10% do treino separados para validação. As redes com FC chegam perto de 100% no treino, enquanto a validação fica parada. Nas redes com FC sem BatchNorm, a melhor época costuma vir cedo: são os casos marcados no texto.

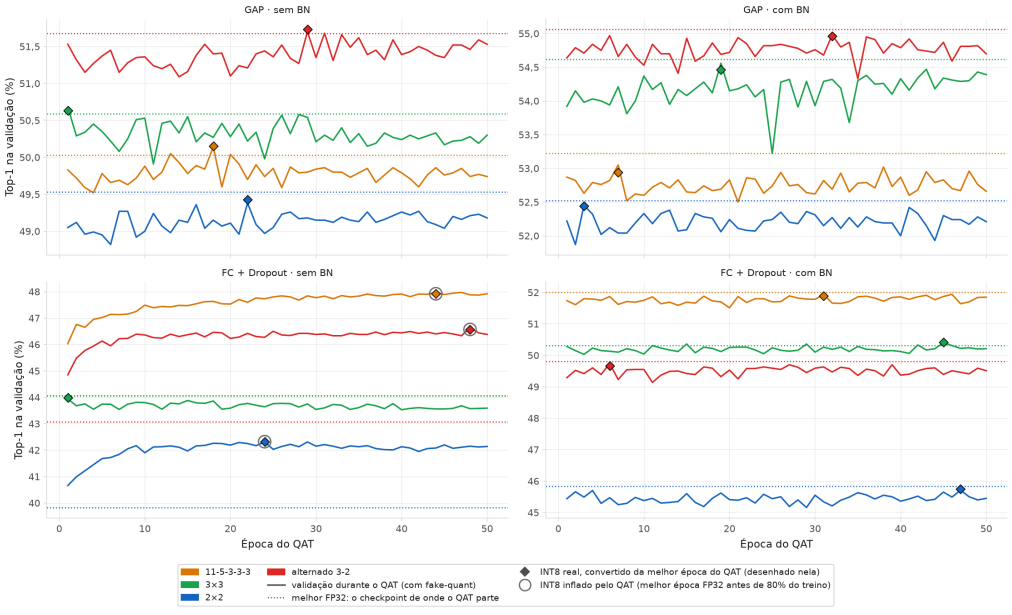


Figura 17. Curvas do QAT nas mesmas redes. O pontilhado é o FP32 de onde o QAT parte, e o losango é a época convertida para INT8. Só as redes com anel sobem claramente acima do ponto de partida: é o treino extra descrito na Seção III-E.



Figura 18. Classificador: FC + Dropout contra GAP, em pares de redes iguais no resto.

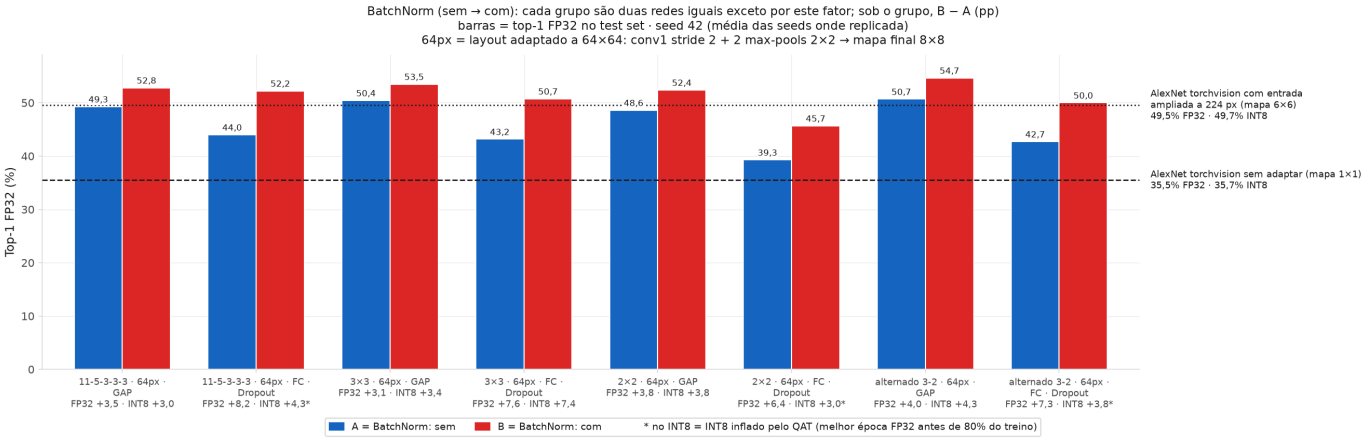


Figura 19. BatchNorm: sem contra com, em pares de redes iguais no resto. O BatchNorm ganha em todos os pares.

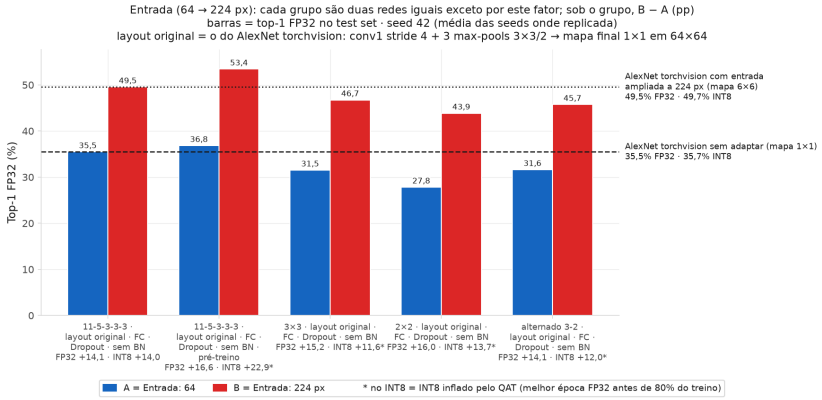


Figura 20. Entrada de 64 px contra a mesma imagem ampliada para 224 px, com o layout original.

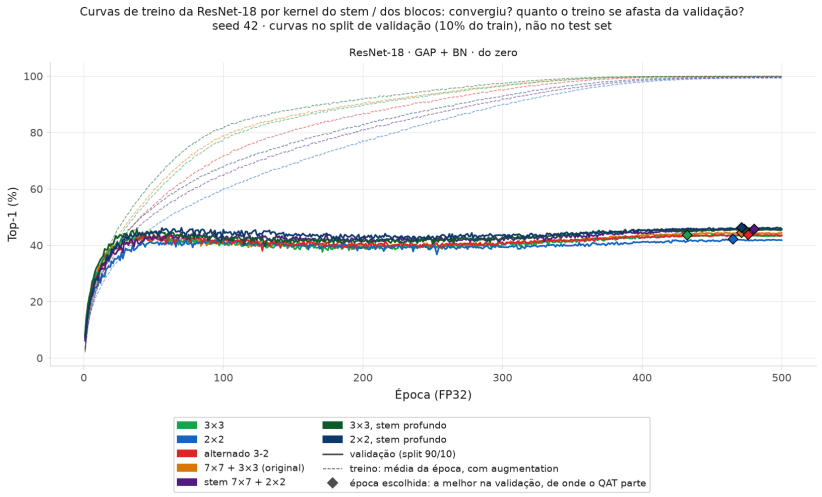


Figura 21. Curvas de treino da ResNet-18, por kernel da primeira camada e dos blocos (seed 42). O treino chega perto de 100%, enquanto a validação fica entre 40% e 46%.

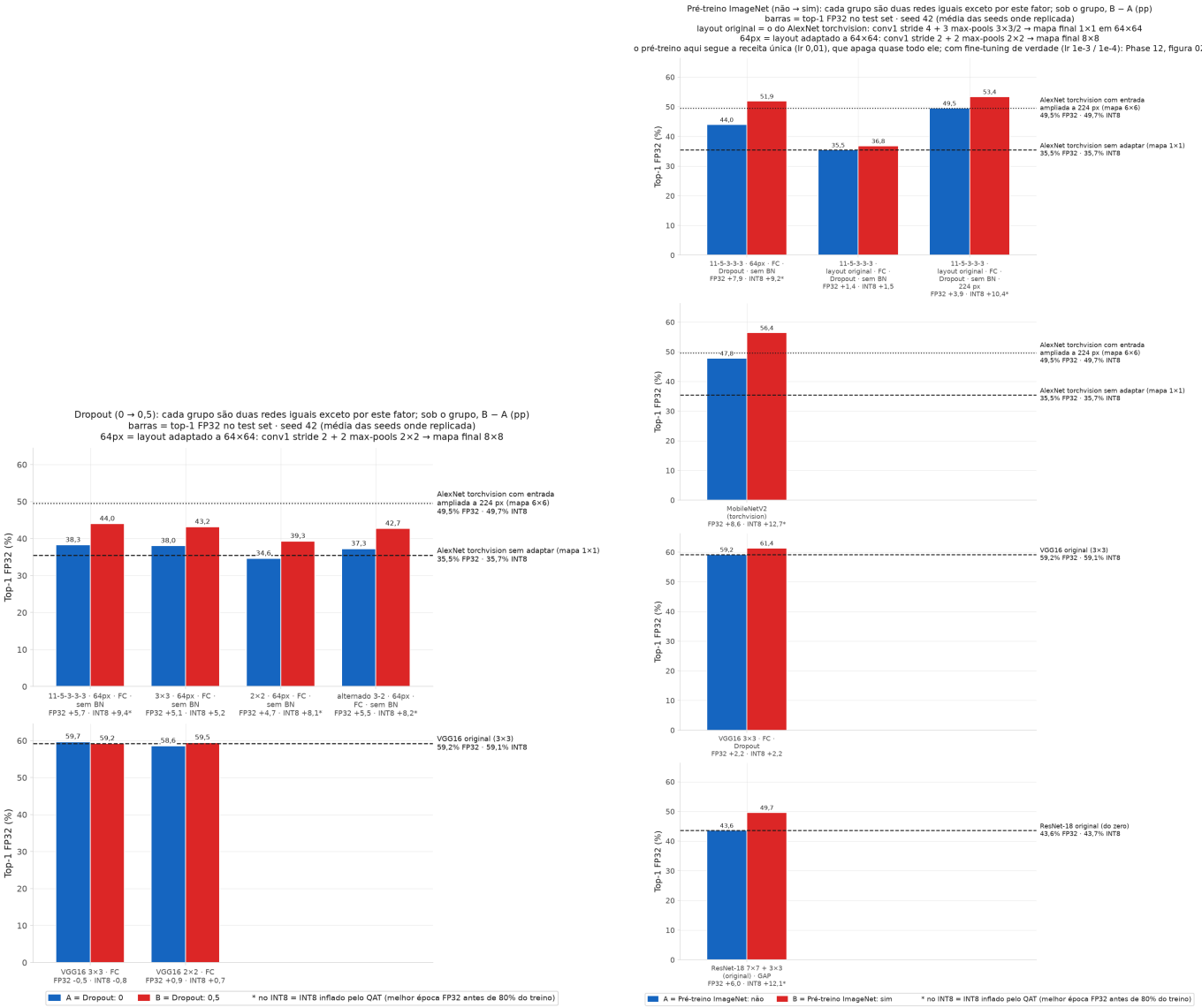


Figura 22. À esquerda, Dropout de 0 a 0,5, só no classificador FC. À direita, pré-treino na ImageNet com a receita única (taxa de aprendizado 0,01).